

Name: K RAJ BHARATH K RAJ BHARATH | Reg. No: RA2511003011548

Unit – 5 : Session – 9 : SLO – 2 | Implementation of Shortest Path Algorithm — Code Activities

■ I. Pseudocode Fill in the Blanks — Dijkstra's

1. Initialize distances of all vertices as INFINITE (INT_MAX), except source = 0.
2. Create a SET (sptSet / visited array) to track visited vertices.
3. For (V-1) times:
 - a. Pick the vertex u with the MINIMUM distance.
 - b. Mark u as visited.
 - c. For all adjacent vertices v: if v not visited and $\text{dist}[u] + \text{weight}(u,v) < \text{dist}[v]$, update $\text{dist}[v]$.

■ II. Complete the C Code Snippet

- `int minDistance(int dist[], bool sptSet[], int V) {`
- `int min = INT_MAX, min_index;`
- `for (int v = 0; v < V; v++)`
- `if (sptSet[v] == false && dist[v] < min)`
- `min = dist[v], min_index = v;`
- `return min_index;`
- `}`

■ III. Bug Hunt — 3 Bugs in the Snippet

- Bug 1: '`count <= V`' runs one extra iteration -> should be `count < V - 1` (or `< V`).
- Bug 2: '`sptSet[u] = false`' un-marks the picked vertex -> should be `sptSet[u] = true`.
- Bug 3: '`sptSet[v] == true`' relaxes only visited vertices -> should be `sptSet[v] == false`.
- Corrected loop:
 - `for (int count = 0; count < V - 1; count++) {`
 - `int u = minDistance(dist, sptSet, V);`
 - `sptSet[u] = true;`
 - `for (int v = 0; v < V; v++)`
 - `if (graph[u][v] != 0 && sptSet[v] == false && dist[u] + graph[u][v] < dist[v])`
 - `dist[v] = dist[u] + graph[u][v];`
 - `}`

■ IV. Choose the Best Data Structure

- Small graph ($V \leq 10$): SIMPLE ARRAY / adjacency matrix — overhead of other structures is not worth it;
- $O(V^2)$ is trivially small.
- Dense graph ($V = 1000$, $E = 900000$): ADJACENCY MATRIX with array-based Dijkstra $O(V^2)$ — E is near V^2 ,
- so heap overhead gains nothing and matrix gives $O(1)$ edge lookup.

- Sparse graph ($V = 1000$, $E = 3000$): ADJACENCY LIST + PRIORITY QUEUE (min-heap) — $O((V+E) \log V)$ is far
- better than $O(V^2)$ when E is small; lists store only existing edges.

■ V. Map the Escape Paths — Paths from vertex 0 to vertex 5

- (Using the session graph: 0-1(2), 0-2(6), 1-3(5), 2-3(8), 3-5(15), 3-4(10), 4-5(6), 5-6(6), 4-6(2))
- Path 0-1-3-5: cost $2 + 5 + 15 = 22$ <- SHORTEST
- Path 0-1-3-4-5: cost $2 + 5 + 10 + 6 = 23$
- Path 0-1-3-4-6-5: cost $2 + 5 + 10 + 2 + 6 = 25$
- Path 0-2-3-5: cost $6 + 8 + 15 = 29$
- Path 0-2-3-4-5: cost $6 + 8 + 10 + 6 = 30$
- Best escape path: 0 -> 1 -> 3 -> 5 with total cost 22.